

SuryaGrid AI

Technical Implementation and Real-Data Workflow Report

Evidence-Based Report on Repository Architecture, Kaggle Data, Agents,
APIs, Frontend, and AWS Hosting

Author: SuryaGrid Repository

Date: 7 July 2026

Hosted URL: <http://suryagrid.mithungowda.in/>

Repositories: DBIT-Bangalore/SuryaGrid, lekhanpro/SuryaGrid

Scope: Verified repository + hosted-app state only

Every claim is backed by an entry in `docs/report/report_evidence_matrix.md` (repository files, generated artifacts, model cards, command output, or the hosted site). Unproven items are marked **Pending**, **Not verified**, **Documented**, or **Not present** and are never asserted as complete. This report does not modify application logic.

Contents

1	Abstract	2
2	Project Objective and Scope	2
3	Verification Method	2
4	Repository and System Overview	3
5	High-Level Architecture	3
6	Data Sources Used	3
7	Kaggle Dataset Pipeline	4
8	Substation and Grid Data Pipeline	5
9	Formula and Calculation Documentation	5
10	Agent Architecture	6
10.1	Data source and API-manager agents	6
10.2	Weather and solar forecasting agents	6
10.3	Cloud risk agent	6
10.4	Location / substation / grid agents	6
10.5	DSM and decision agents	7
10.6	Fuzzy / RL / reward agents	7
10.7	Orchestrator agent	7
11	Agent Workflow	7
12	Backend API Architecture	8
13	Frontend Architecture and API Connection	8
14	ML Training Results	9
15	Hosted AWS Deployment Status	10
16	Testing and Verification	10
17	Current Limitations	11
18	Next Phase Implementation Plan	11
19	Conclusion	12
	References	13

1 Abstract

SuryaGrid AI is a multi-agent solar-forecasting and Deviation Settlement Mechanism (DSM) support platform focused on Bengaluru / Karnataka / India. This report documents the verified state of the repository and the hosted application on 7 July 2026. The hosted site at <http://suryagrid.mithungowda.in/> is live: the frontend returns HTTP 200 (nginx / Next.js) and the backend health endpoint returns HTTP 200 with `environment=production`, `database=connected` (postgresql), and `redis=connected`. The backend (FastAPI) registers 56 `/api/v1` endpoints and passes 103 tests. A real-data Kaggle pipeline was executed: four datasets were downloaded (12 raw files), producing three processed and four ML training parquet files and four trained models with cards — a real Indian PV AC-power model ($R^2 = 0.869$, `REAL_INDIA`), a Bengaluru irradiance model ($R^2 = 0.920$, `REAL_BENGALURU`), a Bengaluru cloud-risk classifier ($F_1 = 0.847$), and an India load model ($R^2 = 0.893$, marked not production-ready). Deployment exists on two paths: a single-instance docker-compose+nginx stack (verified live) and an AWS ECS/Fargate Terraform stack (present in code, not verified deployed; HTTPS/ACM commented out). Local PV generation for Bengaluru, metered Karnataka load, official parsed tariff rupee rates, substation capacity, and a trained RL policy are *not* present and are tracked as pending. The platform is an architecturally complete, hosted prototype; it is not full production-ready.

2 Project Objective and Scope

SuryaGrid AI converts real meteorological data into a solar irradiance/generation forecast (physics via `pvlb`, ML via `scikit-learn`), classifies deviation-settlement risk against a schedule, and exposes results through a FastAPI backend and a Next.js dashboard, with every value traceable to a labelled source. The regional focus is Bengaluru / Karnataka / India, which matters because solar output, tariffs, DSM rules, load shape, and substation topology are region-specific and do not transfer from foreign data. The project distinguishes real data (measured/observed), coordinate-based real data (global API at Bengaluru coordinates), estimated data (derived via a documented formula), and synthetic/demo data (labelled, blocked in real mode). The deployment goal is a hosted full stack; a hosted instance exists (Section 15).

3 Verification Method

Findings derive from: (i) repository file inspection (including three parallel context-gathering passes over deployment, frontend, and formula/agent code); (ii) generated artifacts (parquet, `.pkl`, JSON model cards, build/training manifests); (iii) command output (`pytest`, `ruff`, `docker`, route introspection); and (iv) live checks of the hosted URL via HTTP. Each major claim is recorded in `docs/report/report_evidence_matrix.md` with its evidence type and status. Numeric metrics are quoted verbatim from model cards; endpoints from the running route table.

4 Repository and System Overview

Table 1: Repository component status (Table 1).

Component	Path	Purpose	Status	Evidence
Backend API	backend/app	FastAPI, 56 endpoints	Verified	route table
Agents	backend/app/agents	16 agent modules	Verified	docstrings
Data pipeline	backend/app/data_pipeline	Kaggle ingest (3 scripts)	Verified	files + parquet
ML pipeline	backend/app/ml	build/train/inference	Partial	artifacts present
Data sources	backend/app/data_sources	providers + registry	Partial	OSM/Open-Meteo wired
DSM engine	backend/app/dsm	rule profiles + engine	Partial	framework; rupee rates pending
Frontend	frontend/	Next.js 14 App Router, 13 pages	Verified	build + hosted 200
Kaggle datasets	backend/data/	12 raw, 3 proc, 4 ML parquet	Verified	dir listing
Trained models	backend/models/trained	10 Open-Meteo + 4 Kaggle + rules	Partial	pkl + cards
Model cards	backend/models/metadata	cards + manifests	Verified	JSON present
Tests	backend/tests	pytest suite	Verified	103 passed
Deployment (compose)	docker-compose.yml etc.	single-instance stack	Verified	hosted live
Deployment (AWS ECS)	infra/terraform	Fargate/RDS/S3/CloudFront	Documented	code only, not verified

5 High-Level Architecture

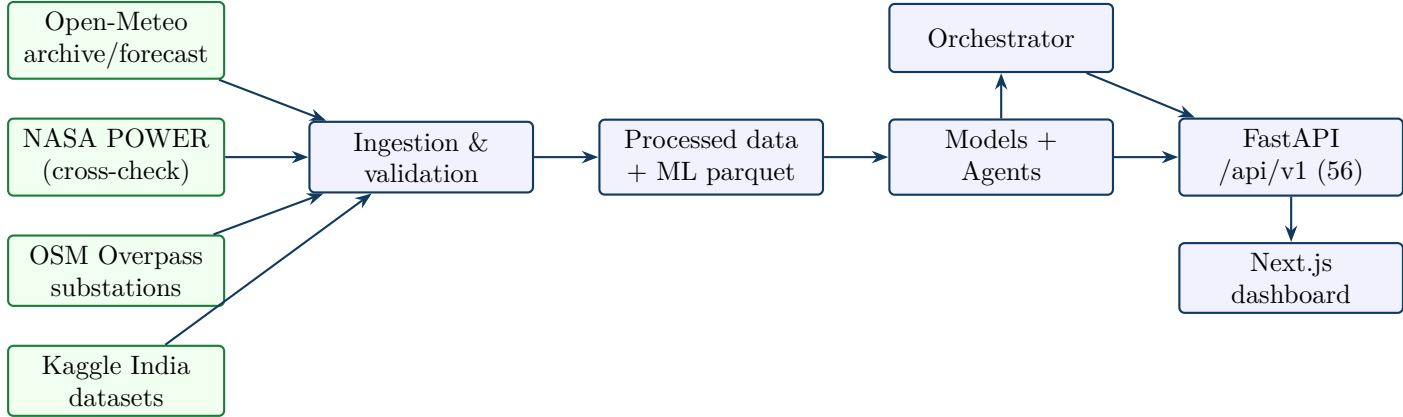


Figure 1: High-level architecture (Figure 1). Green = real external sources. NASA POWER is wired as a build-time cross-check, not a live provider.

6 Data Sources Used

Table 2: Data source status (Table 2).

Source	Type	Geography	Status	Used in	Limitation
Open-Meteo archive	weather	Bengaluru coord	Verified used	weather history (26,304 rows)	reanalysis, not on-site
Open-Meteo forecast	weather	coordinate	Verified wired	live weather agent	fair-use limits

Source	Type	Geography	Status	Used in	Limitation
NASA POWER	weather	Bengaluru coord	cross-check only; provider Pending	build manifest ($r = 0.87$)	not a runtime provider
Kaggle anikannal	PV gen	India	Verified down-loaded/used	PV AC model	34-day, 2 plants, not Bengaluru
Kaggle meenakshi	irradiance	Bengaluru	Verified down-loaded/used	GHI + cloud models	NASA-derived; -999 fill dropped
Kaggle shubham-vashisht	load	India (Nat.+regions)	Verified down-loaded/used	load model	national, not Karnataka
Kaggle arunk-anagolkar	PV gen	unknown	rejected (Not present for training)	—	no times-tamp/geography
OSM / Overpass	substation	Bengaluru/Karn.	Verified used	substations (344)	voltage 41%, capacity 0%
KERC/BESCOM/CEA/ERC/DSM	load	Karn./India	framework Documented ; rates Pending	DSM profiles	no parsed rupee rates
CEA/Grid dia/SLDC	In-load/grid	India/Karn.	Documented only	—	no adapter/code
Synthetic weather	synthetic	n/a	present, blocked in real mode	fallback only	demo only

7 Kaggle Dataset Pipeline

Kaggle search/download uses the Kaggle CLI (10 searches were run). Raw archives are stored under `backend/data/raw/kaggle/{pv_generation,solar,load}/`. Ingestion scripts `backend/app/data_pipeline/ingest_kaggle_{pv_generation,solar,load}.py` read the real files, normalise timestamps/units, stamp source metadata (`is_real=true`, `is_synthetic=false`, `geography`, `source__url`, `quality_score`), and write processed parquet to `backend/data/processed/kaggle/`. `build_kaggle_ml_datasets.py` derives ML training files; `train_from_kaggle.py` trains models with cards. A separate legacy script, `download-kaggle.sh`, targets `dronio/SolarEnergy` (Hawaii HI-SEAS) and is *not* used by the `data_pipeline` scripts.

Table 3: Kaggle dataset pipeline (Table 3).

Slug	Geography	Rows	Cols used	Target	Used for / status
anikannal/solar-power-generation-data	India (REAL_-INDIA)	136,472	irradiation, amb/module temp	ac_power	PV AC model; prod=false
meenakshihihihih/...indian cities	Bengaluru (REAL_-BEN-GALURU)	10,219	calendar, T2M, PS, precip, SZA	ghi_wm2	irradiance model; prod=true
meenakshihihihih/...indian cities	Bengaluru (REAL_-BEN-GALURU)	5,356	calendar, T2M, PS, precip	irradiance_drop_risk	cloud model; prod=true
shubhamvashisht/hourly-load-india	India (REAL_-INDIA)	46,560	calendar, lags, roll	national_demand_mw	load model; prod=false
arunkanagolkar/solar-generation-known	unknown	—	—	—	rejected (no time-stamp/geo)

Raw evidence: Plant_1/2_Generation_Data.csv (real AC_POWER, DC_POWER), Plant_1/2_Weather_Sensor_Data.csv (real IRRADIATION, AMBIENT TEMPERATURE, MODULE TEMPERATURE);Bengluru solar

irradiance.csv (ALLSKY_SFC_SW_DWN=GHI, ALLSKY_KT=clearness; 7,349 of 17,568 rows are the -999 fill value, dropped); hourlyLoadDataIndia.xlsx (National + Southern Region Hourly Demand, MW).

8 Substation and Grid Data Pipeline

Substations are ingested from OpenStreetMap via the Overpass API (app/data_sources/substation_provider.py, ODbL). The build manifest records 344 substations around Bengaluru with voltage present on 41% of records and capacity on 0% (kept null — never invented). Grid features are geometric (distance to city centre, nearest-neighbour, density) derived from real coordinates. No official KPTCL/BESCOM/CEA capacity or feeder mapping is present.

Table 4: Substation / grid data status (Table 4).

File	Source	Records	Missing fields	critical	Status
bengaluru_substations_cleaned.parquet	OSM Overpass	344	capacity (100%), voltage (59%), district (100%)		Partial
bengaluru_grid_features.parquet	derived geometry	344	(n/a — geometry only)		Partial
KPTCL/BESCOM capacity	official	—	all		Not present / Pending

9 Formula and Calculation Documentation

All rows below are quoted from docs/formulas.md, docs/FORMULA_SOURCES.md, docs/DSM_RULE_SOURCES.md, or the cited code. "OFFICIAL" means a standard library (pvlib) or standard unit conversion; "FALLBACK_DEFAULT" and "heuristic" are engineering choices; "PENDING" means an official source is required.

Table 5: Formula / calculation table (Table 5).

Calculation	Expression	Code / doc path	Source class
Clear-sky GHI	pvlib Ineichen at (12.9716, 77.5946), alt 920 m	agent_models.py; formulas.md	OFFICIAL (pvlib)
Clearness index	$k_t = \text{GHI}/\text{clearsky_GHI}$ (daylight; clip 0–1.2)	formulas.md	derived
Cloud drop label	drop = 1 if $k_t < 0.5$ (daylight)	build_kaggle_ml_datasets.py	heuristic (documented)
Irradiance decomposition	Erbs GHI→DNI/DHI	FORMULA_SOURCES.md (pvlib)	OFFICIAL (pvlib)
Plane-of-array	<code>get_total_irradiance</code>	FORMULA_SOURCES.md	OFFICIAL (pvlib)
Cell temperature	Faiman model	FORMULA_SOURCES.md	OFFICIAL (pvlib)
DC power	PVWatts; $\gamma_{pdc} = -0.0035/^{\circ}\text{C}$	FORMULA_SOURCES.md	OFFICIAL/FALLBACK
AC / inverter	PVWatts inverter eff. 0.96	FORMULA_SOURCES.md	FALLBACK_DEFAULT
PV proxy estimate	$(\text{POA}/1000) \cdot \text{cap} \cdot \eta \cdot \text{PR}$, PR=0.80	agent_models.py	FALLBACK_DEFAULT
Confidence	$\text{clamp}(1 - 0.35 c_f, 0.4, 0.99)$	FORMULA_SOURCES.md	FALLBACK_DEFAULT

Calculation	Expression	Path	Class
DSM deviation %	$ actual - scheduled / denom \times 100$; denom = avail. capacity (CERC 6(2)(a)) or scheduled	dsm_engine.py	framework OFFICIAL
DSM band (ML path)	$\pm 15\%$ modelling band	formulas.md	FALLBACK_- DEFAULT
KERC slabs	$\pm 5\%$ band; INR 2/4/6 per kWh slabs	india_dsm_rules.py	band OFFICIAL; rates Pending
DSM charge	$\sum (slab\% \cdot denom \cdot hours) \cdot slab_rate$	dsm_engine.py	formula OFFICIAL; rates Pending
Cyclical time	$\sin / \cos(2\pi h/24)$, doy/365.25, dow/7	build_ml_datasets.py	standard
Load lags	lag_24h, lag_168h, roll_24h_mean	build_kaggle_ml_datasets.py	standard
Unit conversions	F→C, inHg→hPa, mph→m/s	FORMULA_- SOURCES.md	OFFICIAL
Metrics	MAE, RMSE, MAPE, R^2 (sklearn)	train_utils.py	OFFICIAL

Important: no rupee DSM charge is emitted in the Phase 1.7 ML/agent path (it returns bands only, NEEDS_OFFICIAL_TARIFF_SOURCE). Rupee charges are only computed by the dsm/ rule engine using USER_CONFIGURABLE/PENDING placeholder rates; the KERC $\pm 5\%$ band and CERC framework are official, but the numeric rates are not verified.

10 Agent Architecture

Agents are deterministic Python coordinators (no LLM in the numeric path). Sixteen agent modules exist; the Phase 1.7 ML inference layer (app/ml/agent_models.py) serves the trained models behind /api/v1/agents/* and /api/v1/kaggle/.

10.1 Data source and API-manager agents

api_agent.py (provider rotation/failover/quota), api_management_agent.py (health, provider status, rate limiting, retries), kaggle_data_agent.py (download/normalise Kaggle solar), source_registry_agent.py (source records + formula references), feature_engineering_agent.py (augmented dataset + quality), persistence_agent.py (DB writes). Status: **Complete/Partial**.

10.2 Weather and solar forecasting agents

live_weather_agent.py (Open-Meteo, Redis cache, labelled synthetic fallback) and forecast_agent.py (formula/ML/hybrid nowcast). The Phase 1.7 solar model predicts GHI (irradiance), not PV; PV is an explicit estimate. Status: **Partial**.

10.3 Cloud risk agent

Implemented as the classifier cloud_risk_classifier.pkl / kaggle_cloud_risk_bengaluru_model.pkl served via agent_models; label is the real clearness index $k_t < 0.5$. No dedicated cloud_agent.py class file. Status: **Partial**.

10.4 Location / substation / grid agents

location_data_agent.py exposes sites, substations (OSM), nearest mapping, and data coverage; capacity/voltage are kept null where missing. Status: **Partial**.

10.5 DSM and decision agents

`dsm_engine_agent.py` (resolve profile, evaluate deviation, persist), `dsm_classifier_agent.py` (band breach + penalty estimate), and `app/dsm/` (profiles: kerc-solar $\pm 5\%$, cerc-2024-ws-generic $\pm 10\%$, generic-configurable). The DSM engine computes bands; rupee charges use placeholder/pending rates. Status: **Partial**.

10.6 Fuzzy / RL / reward agents

`fuzzy_risk_agent.py` (triangular membership, Mamdani, centroid), `risk_agent.py` (deterministic risk level), `reward_agent.py` (settlement engine), and `app/rl/`. No RL policy is trained (`rl_policy.zip` absent; `card production_ready=false`, `INSUFFICIENT_REAL_ENVIRONMENT_DATA`). Status: **Blocked** for RL.

10.7 Orchestrator agent

`orchestrator_agent.py` sequences Forecast $\rightarrow$ DSM $\rightarrow$ Risk $\rightarrow$ Explanation and coordinates the end-to-end site prediction. Status: **Complete**.

Table 6: Agent architecture (Table 6).

Agent	Category	Model/data used	API connection	Status
<code>api_management_agent</code>	source/mgmt	health/providers	/system/status	Complete
<code>live_weather_agent</code>	forecasting	Open-Meteo + Redis	/weather/*	Complete
<code>forecast_agent</code>	forecasting	solar model + pvlib	/predict, /forecast	Partial
<code>(cloud classifier)</code>	forecasting	cloud_risk_classifier	/agents/cloud/risk	Partial
<code>location_data_agent</code>	grid/location	OSM substations	/substations, /locations	Partial
<code>dsm_engine_agent</code>	DSM/decision	profiles + dsm model	/dsm/*, /agents/dsm/assess	Partial
<code>fuzzy_risk_agent</code>	DSM/decision	deviation+weather	(within predict)	Complete
<code>reward_agent</code>	DSM/decision	settlement scheme	/settle/*, /rl/*	Partial
RL policy	RL	(insufficient data)	/rl/*	Blocked
<code>orchestrator_agent</code>	orchestration	all above	/predict/{id}	Complete

11 Agent Workflow

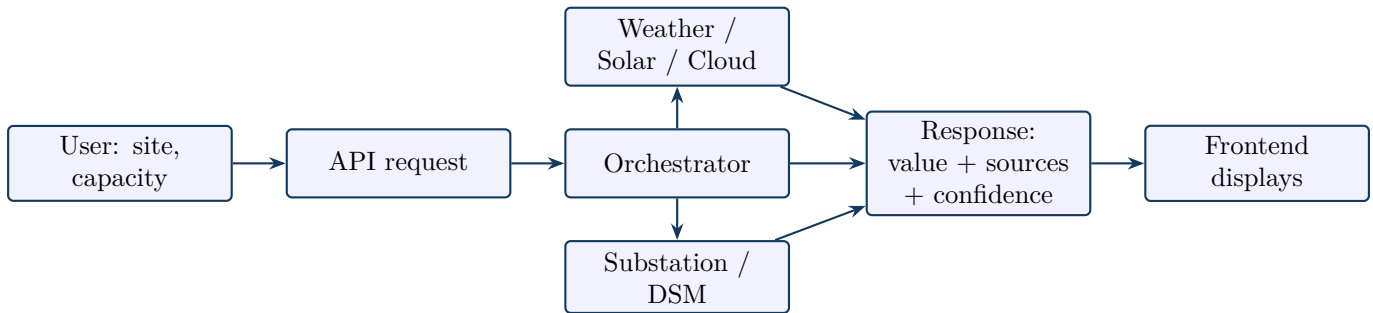


Figure 2: Agent orchestration workflow (Figure 2).

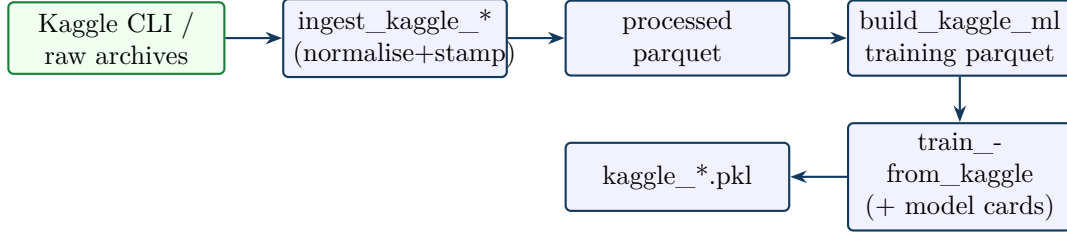


Figure 3: Kaggle data ingestion and training workflow (Figure 3).

12 Backend API Architecture

The backend is FastAPI with routers grouped by domain under `backend/app/api/`, all mounted at `/api/v1`; 56 endpoints are registered. Responses use a standard envelope `{success,message,data,timestamp}`; the `/agents/*` and `/kaggle/*` endpoints add a provenance envelope (model file/version, training/target geography, `source_status`, `production_ready`, limitations). CORS origins include the hosted domain and the instance IP.

Table 7: Representative API endpoints (Table 7; 56 total).

Method	Path (<code>/api/v1</code>)	Agent/service	Provenance?	FE?
GET	<code>/health</code>	app/db	—	yes
GET	<code>/system/status</code>	api_manage- ment_agent	partial	yes
POST	<code>/predict</code> , GET <code>/predict/{id}</code>	orchestrator	yes	yes
GET	<code>/forecast/{id}</code>	forecast_agent	yes	yes
GET/POST	<code>/weather/*</code>	live_weather_- agent	yes	yes
GET	<code>/agents/status</code> , <code>/agents/data-status</code>	agent_models	yes	yes
POST	<code>/agents/solar/forecast</code>	solar model	yes	yes
POST	<code>/agents/cloud/risk</code>	cloud model	yes	yes
POST	<code>/agents/dsm/assess</code>	dsm model	yes	yes
GET	<code>/kaggle/status</code>	kaggle cards	yes	no
POST	<code>/kaggle/pv/estimate</code>	kaggle_pv_ac_- model	yes	no
GET/POST	<code>/dsm/rule-profiles</code> , <code>/dsm/advanced-check</code>	dsm_engine	yes	yes
GET/POST	<code>/substations</code> , <code>/substations/nearest/{id}</code>	location_data_- agent	yes	yes
GET	<code>/sources</code> , <code>/data-sources/status</code>	source_reg- istry_agent	yes	yes
POST	<code>/ml/train</code> , <code>/ml/predict</code> ; GET <code>/ml/model/status</code>	ml pipeline	yes	yes
GET/POST	<code>/rl/rates</code> , <code>/rl/runs</code> , <code>/rl/train</code>	reward/rl	partial	yes

13 Frontend Architecture and API Connection

The frontend is Next.js 14 (App Router; React 18, Tailwind, axios, recharts). It has 13 page routes (`/`, `/dashboard`, `/data-sources`, `/dsm`, `/ml`, `/locations`, `/energy`, `/settlement`, `/karnataka`, `/rl`, `/predictions`, `/system`, `/timeline`) plus components including `ModelProvenancePanel.tsx` and `OfflineBanner.tsx`. The API client (`frontend/lib/api.ts`) derives the base URL as `NEXT_PUBLIC_API_BASE_URL || NEXT_PUBLIC_API_URL || "http://localhost:8000/api/v1"`; `apiFetch` throws on failure and never fabricates data. In docker-compose the frontend build arg is `NEXT_PUBLIC_API_URL=http://suryagrid.mithungowda.in/api/v1`. The UI surfaces provenance/status verbatim (data mode, source registry classifications, production-ready badges, weather-mode, offline banner). No `/kaggle/*` calls exist in the client (Kaggle models are backend-only).

Table 8: Frontend pages / API (Table 8, selected).

Page	Backend API	Data displayed	Status
/dashboard	/predict, /agents/status	forecast + provenance panel	Verified
/data-sources	/sources, /data- sources/status	source registry + classi- fication	Verified
/ml	/ml/model/status	model status	Verified
/locations	/substations, /lo- cations	sites/substations	Verified
/dsm	/dsm/*	DSM check	Verified
/system	/system/status	health/status	Verified
(Kaggle models)	/kaggle/*	—	Not present (not wired in UI)

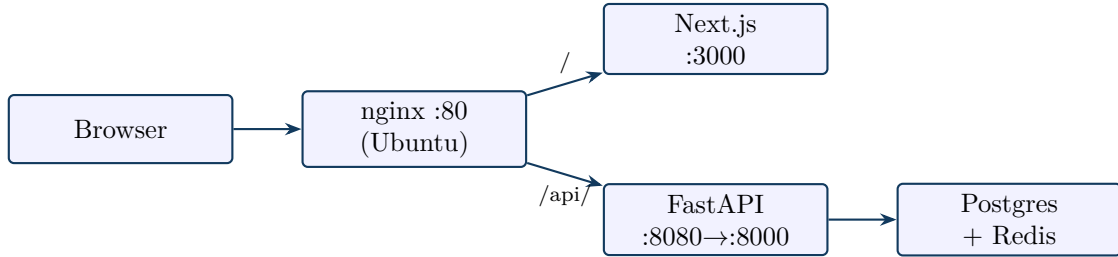


Figure 4: Frontend–backend–API connection on the hosted instance (Figure 4). Verified: / and /api/v1/health both return HTTP 200.

14 ML Training Results

Metrics are quoted verbatim from model cards (chronological 80/20 split).

Table 9: ML model / results (Table 9).

Model	Geography	Metrics	Prod?	Limitation
kaggle_pv_ac_- model	REAL_IN- DIA	$R^2=0.869$, RMSE=118.98 kW	no	India plant, 34d, not Bengaluru
kaggle_solar_irradi- ance_bengaluru	REAL_BEN- GALURU	$R^2=0.920$, RMSE=74.96	yes	GHI only; sec- ondary to Open- Meteo
kaggle_cloud_- risk_bengaluru	REAL_BEN- GALURU	$F_1=0.847$, AUC=0.851	yes	irradiance drop, not PV
kaggle_load_fore- cast_model	REAL_IN- DIA	$R^2=0.893$, RMSE=6404.65 MW	no	national, domain shift HIGH
solar_forecast_- model (Open-Meteo)	REAL_- COORD (Bengaluru)	$R^2=0.956$, RMSE=57.51	yes	irradiance only
cloud_risk_classifier (Open-Meteo)	REAL_- COORD (Bengaluru)	$F_1=0.671$, AUC=0.900	yes	irradiance drop
dsm_classifier (Open-Meteo)	REAL_- COORD (Bengaluru)	$F_1=0.791$, AUC=0.749	yes	no rupees; $\pm 15\%$ band
load_forecast_- model (Open-Meteo)	REAL_IN- DIA	$R^2=0.883$, RMSE=6259.5 MW	no	national, not Karnataka

Model	Geography	Metrics	Prod?	Limit
rl_policy	—	not trained	no	INSUFFICIENT_ REAL_ENVI- RONMENT_ DATA

15 Hosted AWS Deployment Status

The hosted URL <http://suryagrid.mithungowda.in/> was verified live on 7 July 2026. The frontend returns HTTP 200 (Server: nginx/1.24.0 (Ubuntu), X-Powered-By: Next.js). The backend health endpoint `/api/v1/health` returns HTTP 200 with `environment=production`, `database=connected` (postgresql), `redis=connected`, and empty `record_counts`. `/docs` returns HTTP 404 (Swagger not exposed on the hosted path). The site is served over HTTP (port 80); an HTTPS attempt failed because the host’s TLS certificate altname is `apilumi.mithungowda.in`, not the SuryaGrid domain.

The live behaviour matches the single-instance path (`deploy.sh`, `docker-compose.yml`, `nginx.conf`, `setup-subdomain.sh`): nginx proxies `/→frontend:3000` and `/api/→backend:8080` on an Ubuntu instance documented as AWS EC2/Lightsail (IP 32.197.42.115). A separate AWS ECS/Fargate Terraform stack exists in `infra/terraform/` (VPC, ALB, ECS, RDS PostgreSQL 16.3, ElastiCache Redis 7.1, ECR, S3+CloudFront; region `ap-south-1`) driven by `.github/workflows/deploy-aws.yml`, but there is no evidence it is the live deployment; its HTTPS listener and ACM certificate are commented out, and no `terraform.tfvars` exists. The cloud provider (AWS) is documented in `deploy.sh/DEPLOYMENT.md` but not independently confirmable from the HTTP responses.

Table 10: Hosted / deployment status (Table, hosting).

Item	Status	Evidence
Frontend at hosted URL	Verified	HTTP 200, nginx+Next.js
Backend <code>/api/v1/health</code>	Verified	HTTP 200, db+redis connected, prod
Swagger <code>/docs</code> (hosted)	Not present	HTTP 404
HTTPS/TLS for domain	Not verified	cert altname mismatch; HTTP:80 only
Single-instance compose deploy	Verified (behaviour)	matches nginx/compose files
AWS ECS/Fargate (Terraform)	Documented	code only; not verified deployed
Cloud provider = AWS	Documented	<code>deploy.sh</code> / <code>DEPLOYMENT.md</code>

16 Testing and Verification

Table 11: Testing / verification results (Table 10).

Command / check	Result	Notes
<code>pytest tests/ -q</code>	Verified passed	103 1 benign warning
<code>ruff check app tests</code>	Partial	1 error (1 fixable); not fixed (report-only change)
frontend build / hosted	Verified	14 routes earlier; hosted 200 live
route introspection	Verified	56 <code>/api/v1</code> endpoints
hosted /	Verified	HTTP 200 (nginx+Next.js)
hosted <code>/api/v1/health</code>	Verified	HTTP 200, db+redis connected
hosted <code>/docs</code>	Not present	HTTP 404
<code>docker --version</code>	Verified	29.6.1
<code>docker compose version</code>	Not verified	unknown command locally
Kaggle download/ingest	Verified	12 raw, 3 processed, 4 ML parquet

17 Current Limitations

Table 12: Current limitations (Table 11).

Limitation	Status	Impact
Local Bengaluru/Karnataka PV generation	Not present	PV model is REAL_INDIA plant, not local; prod=false
Metered Karnataka load/injection	Not present	load is national India; DSM injection is proxy
Official tariff rupee rates	Pending	DSM emits bands only; charges use placeholders
Substation capacity/voltage	Partial	capacity 0%, voltage 41% (OSM)
AWS ECS stack deployed	Not verified	only single-instance compose verified live
HTTPS on hosted domain	Not verified	HTTP only; cert mismatch
RL policy	Blocked	not trained (insufficient real environment)
Kaggle models in UI	Not present	/kaggle/* not called by frontend

18 Next Phase Implementation Plan

Table 13: Prioritised roadmap (Table 12).

P	Task	Why	Source needed	Acceptance
1	Bengaluru/Karnataka PV generation ingestion	real local PV target	plant/BESCOM feed	PV model REAL_LOCAL, prod=true
2	Karnataka SLDC / Grid India load	localise load + DSM	KPTCL-SLDC / Grid India	load model REAL_KARNATAKA
3	Official KERC/BESCOM tariff parser	real INR charges	KERC/CERC orders	rates OFFICIAL, not placeholder
4	Official DSM rule/penalty validation	correct settlement	CERC 'X' + KERC order	rupee charges verified
5	KPTCL/BESCOM/CEA substation enrichment	capacity/feeder	official grid data	capacity populated
6	Surface Kaggle models + provenance in UI	transparency	(frontend)	/kaggle/* rendered
7	Real-mode enforcement in all routes	no synthetic fallback	(code)	guard on legacy predict path
8	AWS backend/API health monitoring	deployment assurance	CloudWatch/uptime	alerting on /health
9	HTTPS + ACM/Route53 for domain	secure hosting	ACM cert	valid TLS for domain
10	Model monitoring + re-training	drift control	(pipeline)	scheduled retrain + metrics
11	RL environment (after 1-4)	dispatch optimisation	real load+solar+tariff	trained policy on real env

Table 14: Readiness summary (Table 13).

Dimension	Level	Basis
Architecture	Complete	agents + API (56) + frontend + ML pipeline wired
Data	Partial	real weather/solar + India load/PV + substations; local PV/load missing
Kaggle pipeline	Complete	4 datasets ingested, processed, trained, carded
Substation	Partial	344 OSM; capacity/voltage incomplete
ML	Partial	irradiance/cloud/DSM prod-ready; PV/load non-local; RL untrained
DSM	Partial	framework + classifier; official rupee rates pending
Frontend	Complete	Next.js 14, 13 pages, provenance UI, hosted live
Backend/API	Complete	FastAPI, 56 endpoints, 103 tests, hosted health OK
AWS hosting	Partial	single-instance compose live; ECS code-only; no HTTPS
Production	Pending	local PV/load, official tariff, HTTPS, RL outstanding

19 Conclusion

SuryaGrid AI is a working, hosted multi-agent solar/DSM platform. The repository has a verified FastAPI backend (56 endpoints, 103 tests), a Next.js frontend, a real Kaggle data pipeline (four datasets ingested and four models trained with cards), an Open-Meteo Bengaluru forecasting stack, and OSM substation data. The hosted application is live at <http://suryagrid.mithungowda.in/> with both frontend and backend responding and the database and cache connected. It is not full production-ready: local PV generation, metered Karnataka load, official parsed tariff rupee rates, substation capacity, HTTPS, and a trained RL policy are outstanding, and the AWS ECS/Fargate stack is present in code but not verified as the live deployment. These are enumerated in the roadmap (Table 13).

Evidence for every claim: [docs/report/report_evidence_matrix.md](#). Metrics from model cards; endpoints from the route table; hosted status from HTTP checks on 7 July 2026. Report-only change: no application logic was modified.

References

- [1] Fastapi web framework. <https://fastapi.tiangolo.com/>.
- [2] Kptcl state load despatch centre (karnataka). <https://kptclslcdc.in/>. Real-time load despatch/SCADA/energy accounting portal; no clean public API. Not wired.
- [3] Nasa power (prediction of worldwide energy resources). <https://power.larc.nasa.gov/>. Solar/meteorological point API (daily and hourly). Accessed 2026-07.
- [4] Next.js react framework (v14). <https://nextjs.org/>.
- [5] Open-meteo free weather api (forecast and historical archive/era5). <https://open-meteo.com/en/docs>. CC BY 4.0; historical archive endpoint archive-api.open-meteo.com. Accessed 2026-07.
- [6] Openstreetmap overpass api (power=substation). <https://overpass-api.de/api/interpreter>. Data (C) OpenStreetMap contributors, ODbL 1.0. Accessed 2026-07.
- [7] pvlib-python: photovoltaic modelling (erbs, ineichen, faiman, pvwatts). <https://pvlib-python.readthedocs.io/>. BSD-3-Clause.

- [8] scikit-learn: Machine learning in python. <https://scikit-learn.org/>. RandomForest, HistGradientBoosting estimators used.
- [9] Suryagrid ai hosted application. <http://suryagrid.mithungowda.in/>. Frontend + back-end verified live (HTTP 200) on 2026-07-07.
- [10] Anil Kannal. Solar power generation data (two indian plants, 34-day 15-min ac/dc power + irradiation). <https://www.kaggle.com/datasets/anikannal/solar-power-generation-data>. Kaggle dataset; used in this repository as REAL_INDIA PV AC-power training data.
- [11] Central Electricity Regulatory Commission, India. Cerc (deviation settlement mechanism and related matters) regulations, 2024. <https://cercind.gov.in/>. Framework verified; exact “X”/rates set by CERC order (pending).
- [12] dronio. Solar radiation prediction (nasa hi-seas). <https://www.kaggle.com/datasets/dronio/SolarEnergy>. Hawaii HI-SEAS; irradiance target. Registered as REAL_NON_LOCAL / pretraining only.
- [13] Karnataka Electricity Regulatory Commission. Kerc forecasting, scheduling and deviation settlement mechanism framework (karnataka). <https://kerc.karnataka.gov.in/>. Framework verified; exact current tariff/slab order pending.
- [14] meenakshihihihhi. Time series solar irradiance for indian cities (incl. bengaluru; nasa-power-derived hourly). <https://www.kaggle.com/datasets/meenakshihihihhi/time-series-solar-irradiance-for-indian-cities>. Kaggle dataset; Bengaluru extract used as REAL_BENGALURU GHI/clearness.
- [15] Shubham Vashisht. Hourly load india - electrical load forecasting (national + regional demand, mw). <https://www.kaggle.com/datasets/shubhamvashisht/hourly-load-india-electrical-load-forecasting>. Kaggle dataset; REAL_INDIA national demand (Southern Region proxy for Karnataka).
- [16] smarthkaushal. National-level electricity load curve data (india). <https://www.kaggle.com/datasets/smarthkaushal/energy-demand-profile>. Kaggle dataset; “Hourly Demand Met (MW)”, India national aggregate.